Stokes glacier models, with Firedrake

a finite element tutorial

Ed Bueler

University of Alaska Fairbanks

version 2.1 (July 2026)

github.com/bueler/stokes-ice-tutorial

- please ask questions, thanks!
- send questions or comments:
  - by email to `elbueler@alaska.edu`
  - or by using issues at `github.com/bueler/stokes-ice-tutorial/`
- these slides are the primary documentation for some Python codes
- please try the codes, and give me your feedback?

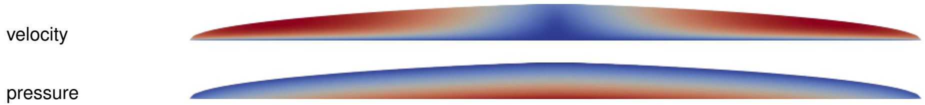
Outline

theory	equations for the glacier dynamics problem
`stage1/`	first demo: linear Stokes
`stage2/`	Glen-Nye-Stokes for glacier ice
`stage3/`	more power: extruded meshes and 3D glaciers
theory	evolving glaciers: free-surface equation coupled to Stokes
`stage4/`	moving-margin dome solutions
	summary and learning more

`github.com/bueler/stokes-ice-tutorial`

glacier dynamics using a Stokes model: the high-level view

- we apply conservation principles to glaciers:
  - mass conservation ✓
  - momentum conservation ✓
  - energy conservation (not here)
- incompressibility is one aspect of mass conservation
  - another is the free-surface equation . . . see `stage4/`
- glaciers are a "slow flow" problem because inertial forces are tiny
  - momentum conservation is therefore called a "stress balance"
  - velocity & pressure are instantaneously determined by glacier geometry and boundary stresses
- Stokes equations = stress balance + incompressibility



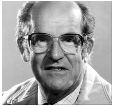
- the canonical glacier dynamics equations are from Glen, Nye, and Stokes
- the main equations:

$$-\nabla\cdot\tau+\nabla p=\rho\mathbf{g} \qquad \textit{stress balance}$$

$$\nabla\cdot\mathbf{u}=0 \qquad \textit{incompressibility}$$

$$\tau=B_n|D\mathbf{u}|^{(1/n)-1}D\mathbf{u} \qquad \textit{Glen-Nye flow law}$$

John Glen



John Nye

  - $\mathbf{u}$ is velocity, p is pressure, τ is the deviatoric stress tensor, and $D\mathbf{u}=\frac{1}{2}\left(\nabla\mathbf{u}+\nabla\mathbf{u}^\top\right)$ is the strain-rate tensor
  - our model has *constant ice density*; incompressibility follows
- constants (*with default values in demos*):
  - $\rho=910\,\text{kg}\,\text{m}^{-3}$
  - $|\mathbf{g}|=9.81\,\text{m}\,\text{s}^{-2}$
  - $n=3$
  - $B_n=6.8\times10^7\,\text{Pa}\,\text{s}^{1/3}$ $\leftarrow$ *constant because isothermal*

viscosity, and avoiding unbounded values

- the classical viscosity relationship $\tau = 2\nu D\mathbf{u}$ eliminates τ from the equations, giving a system for $\mathbf{u}, p$
- however, the unmodified Glen-Nye viscosity formula yields unbounded values as $D\mathbf{u} \to 0$, i.e. for vanishing strain rates
  - note that "$(1/n) - 1$" in $\nu = \frac{1}{2} B_n |D\mathbf{u}|^{(1/n)-1}$ is a negative power
- so we also regularize with small $\epsilon > 0$ to generate bounded viscosity

Glen-Nye-Stokes equations with regularized viscosity

$$-\nabla \cdot (2\nu_\epsilon(|D\mathbf{u}|)D\mathbf{u}) + \nabla p = \rho \mathbf{g} \qquad \textit{stress balance}$$

$$\nabla \cdot \mathbf{u} = 0 \qquad \textit{incompressibility}$$

$$\nu_\epsilon(|D\mathbf{u}|) = \frac{1}{2} B_n \left(|D\mathbf{u}|^2 + (\epsilon\, D_0)^2\right)^{((1/n)-1)/2} \qquad \textit{flow law}$$

- we use $\epsilon = 10^{-4}$ and $D_0 = 1\,\mathrm{a}^{-1}$ in the demonstrations

boundary conditions, as simple as possible

- note that $\sigma = 2\nu_\epsilon D\mathbf{u} - pI$ is the Cauchy stress tensor
- stress-free top:

$$\sigma\mathbf{n} = (2\nu_\epsilon D\mathbf{u} - pI)\,\mathbf{n} = \mathbf{0}$$

    - atmospheric pressure could be applied at surface ... no interesting effect
- no-slip base:

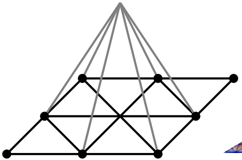
$$\mathbf{u} = \mathbf{0}$$

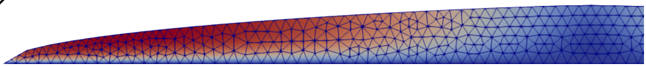
    - no attempt to model basal sliding in this tutorial

speed $|\mathbf{u}|$

finite elements . . . with no details

- by-hand solution of the Stokes model is impossible for most geometries
  - *exercise.* solve slab-on-a-slope, which is an exception
- so we solve numerically using the finite element (FE) method
- the FE method allow us to start from a mesh of triangles (or quadrilaterals, prisms, . . .) over our domain, then express the approximate solution in terms of functions built from the mesh, but I will skip the details . . .





- the whole point of Firedrake is suppress the details of FE!
  - but try Elman, Sylvester & Wathen (2014) for self-study?
- however, one needs to know a key step: write a weak form

the weak form

- start from the equations, now called the *strong form*:

$$-\nabla \cdot (2\nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u}) + \nabla p = \rho \mathbf{g} \tag{1}$$
$$\nabla \cdot \mathbf{u} = 0 \tag{2}$$

- multiply (1) by test velocity $\mathbf{v}$ and (2) by test pressure q, integrate by parts, and combine into one nonlinear residual function:

$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_\Lambda 2\nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u} \,:\, D\mathbf{v} - p\nabla\cdot\mathbf{v} - q\nabla\cdot\mathbf{u} - \rho\mathbf{g}\cdot\mathbf{v}\, dx$$

  - Λ is the spatial domain of ice
  - "dx" $=$ $dx\, dy\, dz$ in 3D, and "dx" $=$ $dx\, dz$ in the planar case

- the actual weak-form problem statement is "residual $= 0$":

$$F(\mathbf{u}, p; \mathbf{v}, q) = 0 \qquad \text{for all test functions } \mathbf{v} \text{ and } q$$

  - the FE method solves the same problem, but over a finite-dimensional space

UFL for the weak form

- nonlinear residual function from last slide:

$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_\Lambda 2\nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u} : D\mathbf{v} - p\nabla \cdot \mathbf{v} - q\nabla \cdot \mathbf{u} - \rho\mathbf{g} \cdot \mathbf{v}\, dx$$

  with $\nu_\epsilon(|D\mathbf{u}|) = \frac{1}{2} B_n \left(|D\mathbf{u}|^2 + (\epsilon\, D_0)^2\right)^{((1/n)-1)/2}$

- we write this using the *Unified Form Language* (UFL) in Firedrake:

```
def D(w):
    return 0.5 * (grad(w) + grad(w).T)

Du2 = 0.5 * inner(D(u), D(u)) + (eps * Dtyp)**2.0
nu = 0.5 * Bn * Du2**((1.0 / n - 1.0)/2.0)

fbody = rho * Constant((0.0, 0.0, - g))
F = ( 2.0 * nu * inner(D(u), D(v)) \
      - p * div(v) - q * div(u) - inner(fbody, v) ) * dx
```

  - Python UFL code is quite similar to the math

- other details we need to know about:
  1. choose separate function spaces for the velocity and the pressure
  2. choose finite element spaces that (the experts say) are stable
- Firedrake makes choosing such mixed elements easy:

```
V = VectorFunctionSpace(mesh, 'CG', 2)  # u space is P2
W = FunctionSpace(mesh, 'CG', 1)        # p space is P1
Z = V * W                               # mixed space
up = Function(Z)
u, p = split(up)
v, q = TestFunctions(Z)
```

  - this $P_2 \times P_1$ choice is called Taylor-Hood
  - other mixed spaces are just as easy
  - we choose $P_2 \times DG_0$ in `stage4/`, for reasons ...

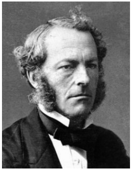
Outline

theory	equations for the glacier dynamics problem
`stage1/`	first demo: linear Stokes
`stage2/`	Glen-Nye-Stokes for glacier ice
`stage3/`	more power: extruded meshes and 3D glaciers
theory	evolving glaciers: free-surface equation coupled to Stokes
`stage4/`	moving-margin dome solutions
	summary and learning more

github.com/bueler/stokes-ice-tutorial

- if we make viscosity constant ($\nu_0 > 0$) then we get a linear Stokes system:

$$\begin{aligned} -\nabla \cdot (2\nu_0\, D\mathbf{u}) + \nabla p &= \rho \mathbf{g} \\ \nabla \cdot \mathbf{u} &= 0 \end{aligned}$$



George Stokes

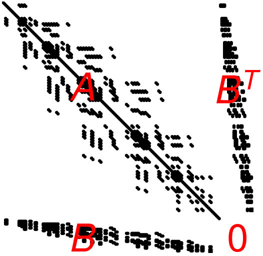
- classical reformat with $2\nabla \cdot D\mathbf{u} = \nabla^2 \mathbf{u}$:

$$\begin{aligned} -\nu_0 \nabla^2 \mathbf{u} + \nabla p &= \rho \mathbf{g} \\ -\nabla \cdot \mathbf{u} \qquad &= 0 \end{aligned}$$

- which has symmetric block structure:

$$\begin{bmatrix} A & B^\top \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix}$$

    - $A = -\nu_0 \nabla^2 \geq 0$
    - $B^\top = (-\nabla \cdot)^\top = \nabla$ (integration-by-parts)



running the demonstration programs

- let's run a demo
- install Firedrake following these directions:

  firedrakeproject.org/install.html

- activate the Python virtual environment every time you use Firedrake:

```
$ source ~/venv-firedrake/bin/activate
```

- other tools you will need:
  - Gmsh gmsh.info
  - Paraview paraview.org
- clone this tutorial, both for the codes and these slides:

```
$ git clone https://github.com/bueler/stokes-ice-tutorial.git
```

- *purpose:* solve linear Stokes on a trapezoidal "glacier"
- *source files:* `domain.geo`, `solve.py` ← inspect these
- *generated files:* `domain.msh`, `domain.pvd`

```
$ cd stage1/
$ gmsh -2 domain.geo                # mesh the domain
$ gmsh domain.msh                   # view the mesh
$ python3 solve.py                  # solve linear Stokes
$ paraview domain.pvd               # view the solution
```

speed $|\mathbf{u}|$

Outline

theory	equations for the glacier dynamics problem
`stage1/`	first demo: linear Stokes
`stage2/`	Glen-Nye-Stokes for glacier ice
`stage3/`	more power: extruded meshes and 3D glaciers
theory	evolving glaciers: free-surface equation coupled to Stokes
`stage4/`	moving-margin dome solutions
	summary and learning more

`github.com/bueler/stokes-ice-tutorial`

Newton's method

- linear Stokes in `stage1/` solves a single linear system:

$$\begin{bmatrix} A & B^\top \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix}$$

  - default solver: Firedrake $\to$ PETSc KSP $\to$ MUMPS
- but Glen-Nye-Stokes residual function is nonlinear in **u**:

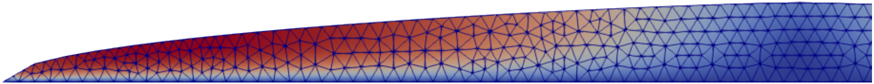
$$F(\mathbf{u}, p; \mathbf{v}, q) = \int_\Lambda 2\, \nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u} : D\mathbf{v} - p\nabla\cdot\mathbf{v} - q\nabla\cdot\mathbf{u} - \rho\mathbf{g}\cdot\mathbf{v}\, dx$$

- fortunately, Firedrake applies Newton's method by default if we write the problem as `solve(F == 0,...)`
- what you need to know about Newton's iteration:
  - it does linearization around each iterate
  - Firedrake uses UFL symbolic differentiation for linearization
  - Firedrake asks PETSc's SNES to do the Newton iteration
  - SNES options will monitor and control the iteration

Isaac Newton

- *purpose:* solve the Glen-Nye-Stokes equations on a dome-shaped glacier with a flat bed
- *source files:* `domain.py`, `solve.py` ← inspect these
- *generated files:* `dome.geo`, `dome.msh`, `dome.pvd`

```
$ cd stage2/
$ python3 domain.py       # generate domain outline
$ gmsh -2 dome.geo        # mesh the domain
$ python3 solve.py        # solve Glen-Nye-Stokes
$ paraview dome.pvd       # view the solution
```



speed $|\mathbf{u}|$

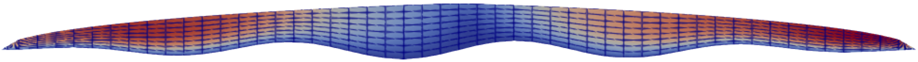
Outline

`github.com/bueler/stokes-ice-tutorial`

a more practical and powerful glacier solver

- this stage takes a more practical and powerful approach
- we switch to an extruded mesh, and allow a bumpy bed (below)
  - the base mesh has lower dimension
  - now the elements are quadrilaterals ($= Q_2 \times Q_1$ mixed method) in the planar case, versus triangles in `stage2/`
- we demonstrate (mostly) dimension-independent programming
  - code for 2D and 3D glacier geometry is nearly the same; elements are prisms in 3D
  - dimension chosen at runtime
- additional bells & whistles:
  - rescaling the equations
  - vertical grid sequencing, with over-regularization on coarse meshes
  - additional diagnostic fields in the output `.pvd` file

solution speed $|\mathbf{u}|$ for default run:

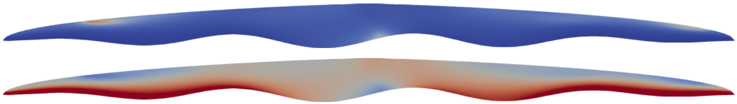


extruded mesh and more

- *purpose:* a practical and robust solver
  - construct mesh by vertically extruding a base mesh — *practical geometry*
  - rescale the equations — *better conditioning*
  - vertical grid sequencing — *better initial iterates*
  - write ν_ϵ, τ, $p - p_{\text{hydrostatic}}$ into output file — *better diagnostics*
- *source file:* `solve.py` ← inspect this
- *generated file:* `dome.pvd`

```
$ cd stage3/
$ python3 solve.py                            # default resolution
$ python3 solve.py -mx 400 -refine 2          # higher resolution
$ paraview dome.pvd
```

effective viscosity ν_ϵ and deviatoric stress magnitude $|\tau|$:



evidence of convergence

- we can now set resolution at runtime, for example:

```
$ python3 solve.py -b0 0 -mx 400 -mz 4 -refine 2
```

  - using optional flat bed (`-b0 0`), to compare results to `stage2/`
  - initial mesh has 400 elements in horizontal and 4 in vertical
  - refine the vertical twice (by factors of four), to generate a 400×64 mesh
- velocity results, in meters per year, from a refinement study:

mesh	$\Delta x \times \Delta z$ (m)	average $\lvert\mathbf{u}\rvert$	maximum $\lvert\mathbf{u}\rvert$
50×8	400×125	1782	3263
100×16	200×63	1766	3211
200×32	100×31	1761	3196
400×64	50×16	1759	3191
800×128	25×8	1759	3190

  - velocities are apparently converging, but we do not know the exact solution . . . this is not verification

parallel computation

- as usual with Firedrake, everything works in parallel
  - extruded mesh gets partitioned following base mesh partition
  - see `rank` field in `.pvd` output; result with 4 processes:

- using the default direct solver (Mumps), results are essentially independent of number of processes
- however, speed-up is sublinear because of significant serial workloads

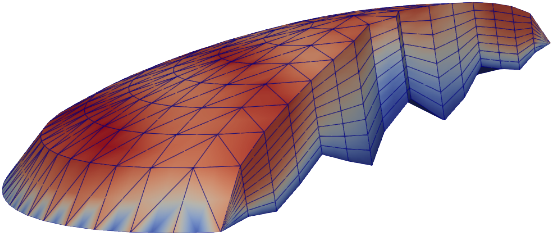
```
$ python3 solve.py -mx 800 -refine 2               # 38 secs run time
$ mpiexec -n 4 python3 solve.py -mx 800 -refine 2  # 20 secs run time
```

- your performance results may vary ...

a 3D glacier

- *purpose:* demonstrate a (mostly) dimension-independent Firedrake code, to model a 3D glacier
- *source file:* `solve.py` ← inspect this
- *generated file:* `dome.pvd`

```
$ python3 solve.py -dim 3 -baserefine 3 -refine 1
$ paraview dome.pvd
```



$3\times$ vertical exaggeration, coloring by speed $|\mathbf{u}|$

solver performance limitations

- this run is about as far as I can refine on my office desktop:

```
$ mpiexec -n 12 python3 solve.py -dim 3 -baserefine 4 \
    -refine 2 -s_snes_atol 1.0e-2 -o finest.pvd
```

- *limiting factor:* the direct solver for each Newton step linear system uses too much memory when generating the LU factors
  - direct solvers experience *fill-in*, especially in 3D; see B (2021b)
- for higher resolution we need a scalable solver
  - such a solver would be built by composition of features: matrix-free, Schur-complement preconditioning, multigrid on the **u**-**u** block; see B (2021b)
  - for a scalable implementation see Isaac, Stadler, & Ghattas (2015)
  - addressing scalability properly is a different talk, but see B (2023)

Outline

theory	equations for the glacier dynamics problem
`stage1/`	first demo: linear Stokes
`stage2/`	Glen-Nye-Stokes for glacier ice
`stage3/`	more power: extruded meshes and 3D glaciers
theory	evolving glaciers: free-surface equation coupled to Stokes
`stage4/`	moving-margin dome solutions
	summary and learning more

`github.com/bueler/stokes-ice-tutorial`

but don't glaciers actually change with the climate?

- we do not yet have a evolving glacier model
  - time derivatives have not appeared!
  - the climate (e.g. surface mass balance) has not appeared as an input

Q. how do glaciers change shape?

A. via the free-surface equation, a kinematical and mass conservation statement:

$$\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s = a$$

  - $z = s(t, x, y)$ is the surface elevation of the glacier
  - $\mathbf{u}|_s$ is the surface value (*trace*) of the ice flow velocity
  - $\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$ is a normal vector to the surface
  - $a(t, x, y)$ is the surface mass balance, the source term

- equation holds on $(0, T) \times \Omega$, where $\Omega \subset \mathbb{R}^d$ is a fixed map-plane region
- initial surface $s(0, x, y)$ is needed

coupled system of equations

- so we have a coupled system of simultaneous equations

coupled free-surface & Glen-Nye-Stokes equations

$$\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s = a \qquad \textit{free-surface equation}$$

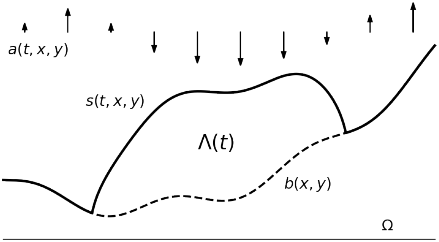
$$-\nabla \cdot (2\nu_\epsilon\, D\mathbf{u}) + \nabla p = \rho \mathbf{g} \qquad \textit{stress balance}$$

$$\nabla \cdot \mathbf{u} = 0 \qquad \textit{incompressibility}$$

- 2 coupling directions:

I. the free-surface equation has $\mathbf{u}|_s$ as a coefficient

II. $s(t,x,y)$ determines the evolving domain $\Lambda(t)$ on which the Stokes problem is solved at each instant



weak form of coupled system of equations

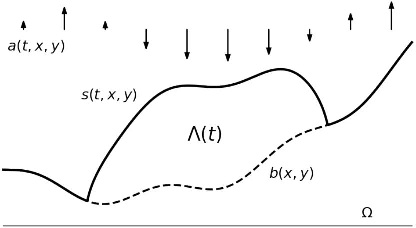
- we get the weak form of the coupled system in the usual way ... multiply by test functions and integrate by parts:

$$\int_\Omega \left(\frac{\partial s}{\partial t} - \mathbf{u}|_s \cdot \mathbf{n}_s - a\right) r\, dx = 0 \qquad \forall r$$

$$\int_{\Lambda(t)} 2\,\nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u} : D\mathbf{v} - p\nabla\cdot\mathbf{v} - \rho\mathbf{g}\cdot\mathbf{v}\, dx = 0 \qquad \forall \mathbf{v}$$

$$\int_{\Lambda(t)} -(\nabla\cdot\mathbf{u}) q\, dx = 0 \qquad \forall q$$

- high-level choices to make about the coupled system:
  1. time-stepping: explicit or implicit?
  2. solved together (*monolithic*) or split?
  3. spaces and elements for $s, \mathbf{u}, p$?



- early/naive approaches to solving this coupled problem have been fragile
- the method everyone tries first is to solve Glen-Nye-Stokes for the current geometry, then advance the surface, and repeat
  - This is an *explicit* approach on *splitting* the coupling.
  - Resulting performance is stability limited, but in a manner which is not precisely understood.

- what are the underlying challenges in designing better schemes?

1. Because s determines the evolving domain $\Lambda(t)$ on which the Stokes problem is solved, the geometry of the mesh used for the $\mathbf{u}, p$ solver changes at every time step. This makes monolithic and/or implicit implementations difficult. (But see Ahlkrona et al (2026).)
2. The coupled problem mixes advective and diffusive aspects, with proportions depending on the particular boundary conditions.
   - Precise time-step restrictions remain unclear.
   - Expected performance scaling remains unclear (B 2023).
   - However, `stage4/` will apply some recent theoretical progress!

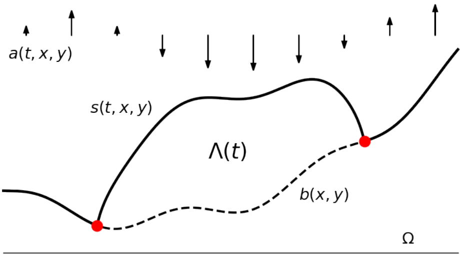
challenges in solving the coupled system

- early/naive approaches to solving this coupled problem have been fragile
- the method everyone tries first is to solve Glen-Nye-Stokes for the current geometry, then advance the surface, and repeat
  - This is an *explicit* approach on *splitting* the coupling.
  - Resulting performance is stability limited, but in a manner which is not precisely understood.
- what are the underlying challenges in designing better schemes?

1. Because s determines the evolving domain $\Lambda(t)$ on which the Stokes problem is solved, the geometry of the mesh used for the $\mathbf{u}, p$ solver changes at every time step. This makes monolithic and/or implicit implementations difficult. (But see Ahlkrona et al (2026).)
2. The coupled problem mixes advective and diffusive aspects, with proportions depending on the particular boundary conditions.
   - Precise time-step restrictions remain unclear.
   - Expected performance scaling remains unclear (B 2023).
   - However, `stage4/` will apply some recent theoretical progress!

also, glaciers have moving margins!

- the surface elevation must be above the bed
- enforcing this obvious fact provides the (lateral) free-boundary condition for a land-based glacier
  - the weak form is a variational inequality (B 2021b, Chapter 12)
  - the thickness and flux are both zero at the free boundary
- solving the coupled problem subject to this condition determines the location of the moving glacier margin ($=$ free boundary)



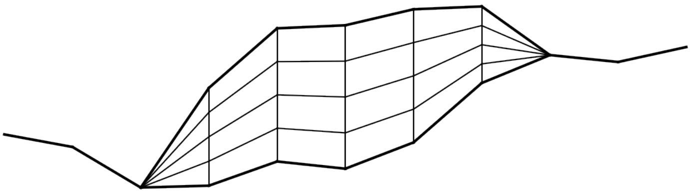
Outline

theory	equations for the glacier dynamics problem
`stage1/`	first demo: linear Stokes
`stage2/`	Glen-Nye-Stokes for glacier ice
`stage3/`	more power: extruded meshes and 3D glaciers
theory	evolving glaciers: free-surface equation coupled to Stokes
`stage4/`	moving-margin dome solutions
	summary and learning more

`github.com/bueler/stokes-ice-tutorial`

evolving extruded meshes for `stage4/`

- we will demonstrate and test a solver which is semi-implicit, split, stabilized, and free margin
  - . . . as explained on the next few slides
  - note that "stabilized" means several things here
- it uses an extruded mesh between continuous P_1 surfaces s and b
  - thus the Stokes domain $\Lambda(t)$ is polygonal at each t



- as before, the extruded mesh for $\Lambda(t)$ is built from quadrilaterals for planar, or prisms for 3D
  - but we will rethink the pressure space

new techniques for `stage4/`

- our solver is based on three new techniques which modify the weak form of the coupled problem:

1. first-order, and mostly explicit, time-stepping
   A. but we add an implicit edge stabilization in the free-surface equation
   B. and we determine the time step by a CFL condition
2. alternating free-surface and Stokes solves
   - but we add a load stabilization coupling to the Stokes solver
3. the coupled free-surface problem is solved as a variational inequality which determines the glacier extent and margin
   A. we apply a bounds-respecting Newton method
   B. we allow variable ice extent on the extruded mesh
   C. we prefer pressure elements with interior degrees of freedom

- the next slides give citations and details on these techniques

technique 1A: implicit edge stabilization in free-surface equation

- this technique is from Tominec et al (2026)
  - see also Burman & Hansbro (2004), Ern & Guermond (2013)
- add FE edge stabilization, applied semi-implicitly:

$$\int_{\Omega^h} \left(\frac{s_{k+1}^h - s_k^h}{\Delta t} - \mathbf{u}|_{s_k^h} \cdot \mathbf{n}_{s_k^h} - a \right) r^h \, dx + J(s_{k+1}^h, r^h) = 0$$

for all test functions r^h, where

$$J(s,r) = \sum_{e \in E^h} \int_e \frac{1}{2} h_e^2 \left| \mathbf{u}|_{s_k^h} \right| [\![\nabla s]\!] [\![\nabla r]\!] \, dS$$

  - s_k^h is the FE approximation of surface elevation at time t_k
  - E^h is the set of interior edges in the triangulation Ω^h
  - $[\![\mathbf{v}]\!] = \mathbf{v}_+ \cdot \mathbf{n}_+ - \mathbf{v}_- \cdot \mathbf{n}_-$ is the jump of a vector field along an edge
  - h_e is the average of the diameters of the two cells which meet e
  - note surface velocity and slope are evaluated explicitly, at t_k
  - the system to solve for s_{k+1}^h is linear

technique 1B: CFL determination of time step

- our application has an advancing glacier margin, which is not considered in the proof of stability in Tominec et al (2026)
- note that the edge stabilization technique applies semi-implicitly, with velocity evaluated explicitly
- the time step Δt must be limited by a CFL[1] criterion:

$$\Delta t = C_{\text{CFL}} \frac{\min h}{\max |\mathbf{u}|}$$

  - margin advance limited to one (base mesh) element per time step
  - we get reasonable results with $C_{\text{CFL}} = 0.25$
    - Ern & Guermond (2013) use $C_{\text{CFL}} = 0.2$ or 0.25 with weighted edge stabilization and SSP3, RK4 schemes

[1] = Courant-Friedrichs-Lewy

technique 2: semi-implicit coupling heuristic in stress balance

- also from Tominec et al (2026), but based on Löfgren et al (2022)
- add extrapolated surface mass balance load to the Stokes form:

$$\int_{\Lambda(t_k)} 2\,\nu_\epsilon(|D\mathbf{u}|)\, D\mathbf{u} : D\mathbf{v} - p\nabla\cdot\mathbf{v} - \rho\mathbf{g}\cdot\mathbf{v}\,dx + \int_{\Gamma_s(t_k)} \rho g \Delta t \left[\frac{1}{2}(\mathbf{u}|_{s_k}\cdot\mathbf{n}_{s_k}) + a\right](\mathbf{v}\cdot\mathbf{n})\,ds = 0$$

  - $\Lambda(t_k)$ is the Stokes domain defined by the current surface elevation s_k
  - $\Gamma_s(t_k)$ is the upper surface of $\Lambda(t_k)$
  - $\mathbf{n}_{s_k} = \langle -\nabla s_k, 1\rangle$ is an outward normal on $\Gamma_s(t_k)$
  - $\mathbf{n} = \mathbf{n}_{s_k}/|\mathbf{n}_{s_k}|$ is the outward *unit* normal

- concept: modifying the stress balance stabilizes the coupling
  - Reynold's transport theorem motivates this stabilization
  - ...but the above form is additionally symmetrized
  - Tominec et al (2026) prove that techniques 1A & 2 together stabilize the L^2 norm of the evolving surface

`stage4/` slumping in a box

- *purpose:* exhibit numerical stability and exact mass conservation for an evolving surface between lubricated walls
  - this geometry is considered by Löfgren et al (2022) and Tominec et al (2026)
- *source files:* `geometry.py`, `halfar.py`, `solve.py` ← inspect these
- *generated file:* `dome.pvd`

```
$ cd stage4/
$ python3 solve.py -walls -L 8000
$ paraview dome.pvd
```

$t = 0$ a: 

$\rightarrow$

$t = 5$ a: 

instability showing need for stabilization technique 2

- *without* technique 2, the maximum size of the stable time step is significantly restricted (Löfgren et al 2022)
  - runs produce "sloshing" and Stokes solver failures
- to demonstrate what happens *without* technique 2, for each resolution in this table we started with $\Delta t = 1$ a, and reduced Δt by a factor of two until the 5 a run completes without a solver failure

Δx (m)	stable Δt (a) w/o technique 2
640	0.25
320	0.25
160	0.125
80	0.125
40	0.125

  - to produce table: `$ python3 solve.py -walls -L 8000 -noload -nocfl -mx MX -maxdt DT`
    with `MX`$=$ 25, 50, 100, 200, 400
  - Tominec et al (2026) does more experimentation

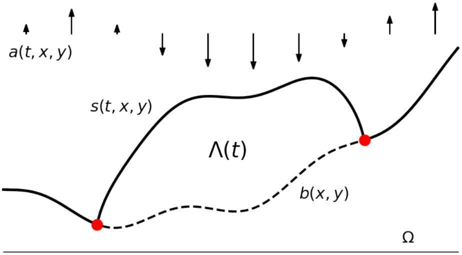
- *with* technique 2, arbitrary time steps are stable w.r.t. sloshing

technique 3A: variational inequality for free surface

- define the FE admissible set $\mathcal{K}^h = \left\{ r^h \in \mathrm{P}_1 \,:\, r^h \ge b^h \right\}$
  - the P_1 bed elevation function b^h is time-independent
- the new surface $s_{k+1}^h \in \mathcal{K}^h$ solves this variational inequality weak form:

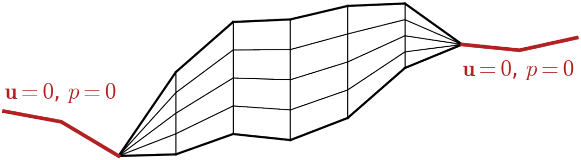
$$\int_\Omega \left(\frac{s_{k+1}^h - s_k^h}{\Delta t} - \mathbf{u}|_{s_k^h} \cdot \mathbf{n}_{s_k^h} - a \right) (r^h - s_{k+1}^h)\, dx + J(s_{k+1}^h, r^h - s_{k+1}^h) \ge 0$$

  for all r^h in $\mathcal{K}^h$
  - includes edge stabilization
- the computed margin location is where s_{k+1}^h meets b^h
- we apply the `vinewtonrsls` bounds-respecting Newton solver from PETSc (B 2021b)



technique 3B: extruded mesh with variable ice extent

- the base mesh covers all of Ω, including areas with no ice
- the extruded mesh has a fixed number `mz` of levels in the vertical
  - $\mathbf{u}, p$ are defined at each node of the extruded mesh
  - our meshes and fields have constant memory footprint as we time-step
- concept: if $s_k^h = b^h$ at a base mesh node then there is no ice in that column of the extruded mesh
  - note that s_k^h and b^h define the ice domain $\Lambda(t_k)$
- in columns with no ice we lock $\mathbf{u}, p$ values to zero
  - classes `PinchColumnPressure|Velocity` are sub-classes of `DirichletBC`; see `stage4/geometry.py`



technique 3C: pressure elements with interior degrees of freedom

- as on previous slide, in columns with no ice we lock **u**, p values to zero
- however, we do not want to lock p to be zero anywhere in the closest-to-margin elements containing ice
  - doing so is an "accidental" Dirichlet condition on the pressure
- thus we prefer elements for pressure which only have interior degrees of freedom, for example:

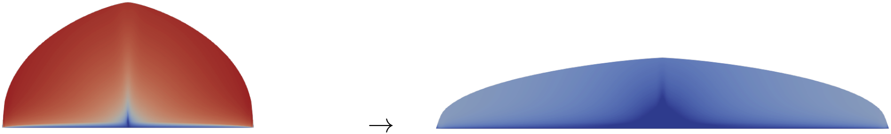
```
W = FunctionSpace(mesh, "DQ", 1, variant="spectral")  # best

W = FunctionSpace(mesh, "DG", 0)  # adequate alternative
```

- for such elements the moving-margin mass conservation error (later slides) is one-sided

a dome flowing outward

- *purpose:* evolving free surface with a moving margin, exhibiting numerical stability
- *source files:* `geometry.py`, `halfar.py`, `solve.py` ← inspect these
- *generated file:* `movie.pvd`

```
$ python3 solve.py -mx 200 -mz 10 -T 100 -L 20000 \
    -omovie movie.pvd
$ paraview movie.pvd
```



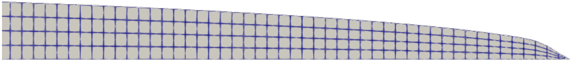
$t = 0$ a → $t = 100$ a

10× vertical exaggeration, $\log(|\mathbf{u}|)$ coloring

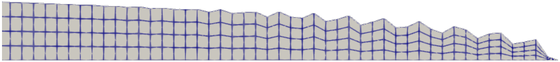
instability showing need for stabilization technique 1A

- especially near the glacier margin, edge stabilization (technique 1A) is clearly needed
- compare these runs:

```
$ python3 solve.py -mx 200 -T 1
```



```
$ python3 solve.py -mx 200 -T 1 -noedge
```

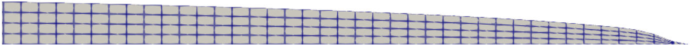


- runs with `-walls` generally do not show the need

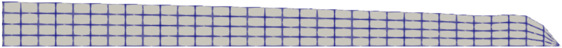
instability showing need for stabilization technique 1B

- need for technique 1B (CFL for time-step length) is more subtle
- compare the margin shape at the end of these two runs:

```
$ python3 solve.py -mx 100
```



```
$ python3 solve.py -mx 100 -nocfl -maxdt 1.0
```



  - advancing margin shape without CFL-limited time steps is wrong
    - bulging and not downsloping
    - not close to Halfar margin location (next slides)
  - but the margin shape is not wildly unstable

Q. is our time-dependent solver correct?

- it is not clear how to build a non-trivial moving-margin exact solution of the coupled equations
- however, such exact solutions exist for shallow ice approximation (SIA)
  - both time-dependent (Halfar) and steady-state exact solutions are known
- SIA exact solutions are close to solving the free-surface + Stokes coupled equations when the aspect ratio $\epsilon = (\text{height})/(\text{width})$ is small

- next slide: we compare with the Halfar (1981) exact solution
  - on a shallow initial dome geometry, so SIA is reasonably accurate
  - initial height 1000 m and width 20 km $\implies$ $\epsilon = 0.05$

evidence of correctness for a shallow, moving-margin dome

- consider runs of 10 years at different horizontal resolutions:

```
$ python3 solve.py -mz 10 -T 10 -mx MX
```

  - use `MX`$=50, 100, 200, 400, 800$, giving $75 \le \Delta x \le 1200$ m
  - from 33 (`MX`$=50$) to 557 (`MX`$=800$) adaptive-scheme time steps
  - max computed ice speed 4600 m/a initially; 160 m/a for final state
- compare final computed geometry to Halfar's prediction:

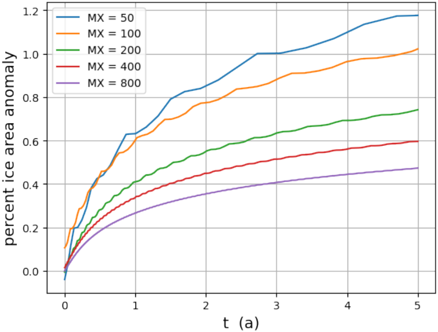
Δx (m)	width (km)	center height (m)
600	28.800	697.789
300	28.800	697.026
150	28.800	696.107
75	28.950	695.670
37.5	28.875	695.329
Halfar	28.993	689.829

- reasonably close ... we do *not* expect convergence to the Halfar values

mass conservation errors with moving glacier margins

- since $a = 0$ here, mass is exactly conserved in the continuum model

- unfortunately, our scheme does not conserve mass here
- numerical mass ($=$ ice area) drifts about 1% over 5 years
  - curves should be horizontal lines
  - higher res $\implies$ better conserve



- this is only a moving margin effect
  - our scheme conserves mass for fixed walls (proved by Tominec et al 2026)
  - for this moving-margin, $a = 0$ case mass is non-decreasing (proved from VI)
- glacier margin mass conservation issues addressed in B (2021a) and Jarosch et al (2013) are distinct from issue here

Outline

`github.com/bueler/stokes-ice-tutorial`

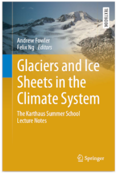
summary and outlook

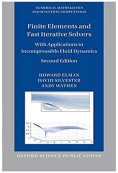
- Firedrake makes the solution of the glacer Stokes problem easy
  - the Python codes in this tutorial are short
  - UFL (for weak forms) and PETSc (for nonlinear solvers) are key in the stack
- extruded meshes make sense because glaciers are (generally) thin flows
- major remaining challenges:
  1. make the Stokes solver faster by replacing the direct linear algebra with a scalable matrix-free solver
  2. make time-stepping glacier simulations mass conserving even in free-boundary cases
  3. make the simulation faster by combining the free-surface equation and the Stokes solver in a monolithic implicit step

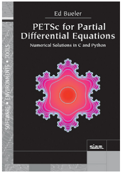
github.com/bueler/stokes-ice-tutorial

- Firedrake: firedrakeproject.org
- PETSc: petsc.org

- for Glen-Nye-Stokes eqns, see Ch. 1 by Hewitt: Fowler & Ng (2021)
- for finite elements and Stokes: Elman, Silvester, & Wathen (2014)
- for PETSc, Firedrake, and Stokes: Bueler (2021b)







references

- J. Ahlkrona, A. C. J. Henry, and A. Löfgren (2026). *A fully implicit second order method for viscous free surface Stokes flow . . .*, Geosci. Model Dev. 19 (6), 2333–2348 doi:10.5194/gmd-19-2333-2026
- E. Bueler (2021a). *Conservation laws for free-boundary fluid layers*, SIAM J. Appl. Math. 81 (5), 2007–2032 doi:10.1137/20M135217X
- E. Bueler (2021b). PETSc for Partial Differential Equations: Numerical Solutions in C and Python, SIAM Press
- E. Bueler (2023). *Performance analysis of high-resolution ice-sheet simulations*, J. Glaciol. 69 (276), 930–935 doi:10.1017/jog.2022.113
- E. Burman & P. Hansbo (2004). *Edge stabilization for Galerkin approximations of convection-diffusion-reaction problems*, Comput. Methods Appl. Mech. Engrg. 193, 1437–1453 doi:10.1016/j.cma.2003.12.032
- H. Elman, D. Silvester, & A. Wathen (2014), Finite Elements and Fast Iterative Solvers, With Applications in Incompressible Fluid Dynamics, 2nd ed., Oxford University Press
- A. Ern & J.-L. Guermond (2013). *Weighting the edge stabilization*, SIAM J. Numer. Analysis 51 (3), 1655–1677 doi:10.1137/120867482
- A. Fowler & F. Ng, ed. (2015). Glaciers and Ice Sheets in the Climate System: The Karthaus Summer School Lecture Notes, Springer Press
- P. Halfar (1981). *On the dynamics of the ice sheets*, J. Geophys. Res. 86 (C11), 11065–11072
- T. Isaac, G. Stadler, & O. Ghattas (2015). *Solution of nonlinear Stokes equations discretized by high-order finite elements . . .*, SIAM J. Sci. Comput. 37 (6), B804–B833 doi:10.1137/140974407
- A. H. Jarosch, C. G. Schoof, & F. S. Anslow (2013). *Restoring mass conservation to shallow ice flow models over complex terrain*, Cryosphere 7 (1), 229–240 doi:10.5194/tc-7-229-2013
- A. Löfgren, J. Ahlkrona & C. Helanow (2022). *Increasing stable time-step sizes of the free-surface problem arising in ice-sheet simulations*, J. Comput. Phys.: X 16 (100114) doi:10.1016/j.jcpx.2022.100114
- I. Tominec, L. Lundgren, A. Löfgren, & J. Ahlkrona (in press). *Free-surface Stokes problem: stability estimates and time-step improvements*